AeroGlide — Code Defense & System Reviewer

Comprehensive Technical Guide & Professor Bensing Defense Reviewer

1. System Overview & Technology Stack

AeroGlide is a full-featured, pure PHP domestic flight and travel package booking platform built for the Philippines market. It features seamless booking flows, promo coupon engine, boarding pass ticket view, flight cancellation with automated refund calculation, and an administrator dashboard.

Component	Technology / Implementation Details
Backend Engine	Pure PHP 8.x with Procedural and Functional Architecture
Database	MySQL (InnoDB, UTF-8 utf8mb4_unicode_ci) via PHP Data Objects (PDO)
Frontend UI	HTML5, Vanilla CSS3 (Custom Design System), Vanilla JS (ES6+), Google Fonts
Web Server	Apache Web Server on WAMP Server Environment
Security & Auth	PDO Prepared Statements, BCRYPT Password Hashing, Session Regeneration, XSS Escaping

2. Modular Directory Architecture

The project follows a clean, modular folder organization separating core user routes, database scripts, destination pages, admin tools, and layout components:

Directory / File	Purpose & Included Functionality
user/	User portal scripts: auth.php, auth_helper.php, login.php, signup.php, logout.php, forgot_password.php, reset_password.php, my_bookings.php
database/	Database layer: config.php (PDO setup), function.php (global helpers), validation.php (Prof Bensing validation), db_setup.php, database.sql
destinations/	Dedicated package pages: albay.php, boracay.php, bukidnon.php, chocolate-hills.php, coron.php, malapascua-island.php, siargao.php, taal-volcano.php
admin/	Admin panel: index.php, bookings.php, users.php, revenue.php, logs.php, guard.php, sidebar.php, booking_detail.php
includes/	Reusable HTML layout parts: head.php, navbar.php, footer.php
Root Wrappers	Root forwarders (index.php, destination.php, checkout.php, help.php, etc.) for clean URL resolution

3. Database Schema & Professor Bensing Validation Rules

A. Database Connection (PDO): Managed in database/config.php using PDO with ERRMODE_EXCEPTION and FETCH_ASSOC. Function ag_db() maintains a singleton PDO instance across requests.

B. Professor Bensing's Validation Standards: Embedded in database/validation.php:

• **Login Input Validation**: validateLoginInput(array $post): array returns ['errors' => [...], 'data' => [...]].

• **Registration Input Validation**: validateSignupInput(array $post): array validates full name, email, username length (3-30 chars), and password length (min 6 chars).

• **Error Handling Standard**: Errors are returned as array, imploded into single query messages (?status=error&message;=...), and rendered in styled alert boxes.

4. Flight Cancellation & Automated Refund Engine

AeroGlide implements an automated trip cancellation and refund calculation workflow in user/my_bookings.php and user/auth_helper.php:

1. **Passenger Selection**: User clicks **[■ Cancel Trip]** on any active booking in My Trips.

2. **Financial Confirmation Warning Modal**: System prompts with a warning modal displaying flight reference code, schedule, passenger name, and full financial breakdown.

3. **Automated Refund & Fee Calculation**:

• **Cancellation Fee**: Fixed at **■500.00** (or capped at total booking amount if lower).

• **Refund Amount**: Calculated as max(0.00, Grand Total - ■500.00).

4. **Accounting Integrity**: Bookings are **never deleted**. The record status updates to CANCELLED, capturing cancellation_fee, refund_amount, and cancelled_at timestamp.

5. Defense Questions & Expert Answers (Professor Reviewer)

Q1: How does your system protect against SQL Injection attacks?

Answer: All database queries in AeroGlide utilize PHP Data Objects (PDO) with prepared statements and parameter binding ($stmt->prepare() and :param bindings). User input is never directly concatenated into SQL strings, completely preventing SQL Injection.

Q2: How is user authentication managed securely?

Answer: Passwords are hashed using BCrypt via password_hash($pass, PASSWORD_BCRYPT). During login, password_verify() authenticates credentials. Upon successful login, session_regenerate_id(true) issues a new session token to prevent session fixation attacks.

Q3: Explain Professor Bensing's validation structure in your code.

Answer: Functions validateLoginInput() and validateSignupInput() in database/validation.php sanitize input and return an associative array containing 'errors' and 'data'. If errors exist, they are imploded into query strings (?status=error&message;=...) and displayed in front-end alert boxes.

Q4: Why do you update booking status to 'CANCELLED' instead of deleting records?

Answer: In real-world enterprise reservation systems, deleting records breaks financial auditability and reporting. We maintain transaction history by setting status = 'cancelled', storing cancellation_fee (■500) and refund_amount, and logging the event in admin_logs.

Q5: How does the promo code engine work and prevent abuse?

Answer: Coupons (e.g., SEPTDEAL2026, 5PEOPLE2026) are validated in database/validation.php. We check minimum guest requirements, compute percentage or fixed deductions, and record used_coupon in the users table to enforce a 1-coupon per account limit.

Q6: How does ag_base_url() handle URL resolution across subdirectories?

Answer: Function ag_base_url() dynamically inspects $_SERVER['SCRIPT_NAME'], strips subdirectories (/user, /database, /destinations, /admin), and returns clean, absolute paths for stylesheets, images, and routes, avoiding broken links.

Q7: How are payment methods stored in the database?

Answer: During checkout, the selected payment method ('gcash', 'card', 'bank') is captured from $_POST['paymentMethod'], validated against an allowed whitelist, and stored in the payment_method column of table bookings.